



Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática

AED III - Trabalho Prático

Documento da Fase V do TP

RELATÓRIO DE CASAMENTO DE PADRÕES (KMP E BOYER-MOORE) E CRIPTOGRAFIA

Este relatório explica como funcionam os métodos de casamento de padrões - KMP (Knuth–Morris–Pratt) e Boyer–Moore (versão com bad character) -, e o método de criptografia. Esse documento foi feito analisando os resultados práticos obtidos no programa desenvolvido.

1. Qual campo textual foi escolhido para aplicar os algoritmos de casamento de padrões?

Por quê?

- **Campo escolhido:** Os atributos *Nome do Estudante* e *Nome do Curso*.
- **Justificativa:** Esses atributos são um campo textual indexável de extrema relevância em sistemas de gestão. Em cenários reais, os administradores raramente buscam usuários exclusivamente por códigos numéricos exatos (ID). Em outros casos, o usuário muitas vezes não lembra o nome completo e digita apenas um sobrenome, um fragmento ou uma palavra-chave. A busca por casamento de sub-strings permite encontrar registros de forma parcial ou aproximada (por exemplo, digitar apenas "Ana" para retornar "Ana Maria" ou "Mariana"). É o campo perfeito para testar a eficiência computacional de algoritmos de busca em arquivos binários de tamanho variável.

2. Explique o funcionamento do KMP implementado

O algoritmo KMP (Knuth-Morris-Pratt) baseia-se no princípio de nunca retroceder o ponteiro do texto principal durante uma busca quando ocorre um descasamento (*mismatch*).

- **Fase de Pré-processamento:** Ele analisa o padrão buscado antes de iniciar a varredura e constrói um vetor auxiliar de falhas chamado LPS (*Longest Proper Prefix which is also Suffix*). Esse vetor mapeia o maior prefixo próprio que também é um sufixo para cada sub-padrão.

- **Fase de Busca:** O algoritmo varre o texto sequencialmente (da esquerda para a direita). Caso um caractere falhe no meio da comparação, o KMP consulta o vetor LPS para saber exatamente para qual índice do padrão ele deve pular, reaproveitando as comparações de caracteres que ele já sabe que deram certo anteriormente. Isso garante um tempo de execução linear de $O(N + M)$, onde N é o tamanho do texto e M o tamanho do padrão.

3. Explique o funcionamento do Boyer-Moore implementado

O algoritmo Boyer-Moore (em nossa implementação utilizando a variação clássica da heurística do Mau Caractere / *Bad Character*) funciona de forma oposta ao KMP no sentido de direção: ele faz o casamento de padrões comparando os caracteres de trás para frente (da direita para a esquerda do padrão).

- **Fase de Pré-processamento:** Cria-se uma tabela indexada (geralmente com o tamanho do alfabeto estendido, 256 posições). Para cada caractere pertencente ao padrão, armazena-se a sua última posição de ocorrência dentro dele. Caracteres que não estão no padrão recebem o valor -1.
- **Fase de Busca:** O padrão é alinhado com o início do texto. A comparação começa pelo último caractere do padrão. Se houver um descasamento:
 1. Se o caractere do texto que causou o erro não existe no padrão, o algoritmo desliza o padrão inteiro para depois desse caractere.
 2. Se o caractere existe no padrão, ele alinha a última ocorrência desse caractere com o texto.
- **Eficiência:** Essa estratégia de saltos longos faz com que, na maioria dos casos práticos, o Boyer-Moore evite ler grandes blocos de texto, tornando-o extremamente veloz (sublinear em cenários ideais).

4. Descreva como integrou os algoritmos ao sistema.

A integração foi feita de forma modular, encapsulando as lógicas em namespaces (BoyerMoore:: e KMP::). Na camada de acesso a dados (DAO) ou na lógica de relatórios, quando o usuário solicita uma pesquisa textual, o sistema lê os registros do arquivo binário e, em vez de usar um simples `==`, invoca a função `contains(textoDoRegistro, stringDeBusca)` de um dos algoritmos. Apenas os registros onde a função retorna `true` são filtrados, adicionados ao vetor de

resultados e listados na interface do usuário.

5. Quais dificuldades encontrou na implementação dos dois algoritmos?

No KMP: A principal dificuldade lógica é a construção matemática do vetor LPS.

Entender o comportamento do cursor `len` que retorna a posições anteriores do próprio prefixo (`len = lps[len - 1]`) requer muito cuidado para não gerar laços infinitos ou acessar memória inválida.

No Boyer-Moore: O maior desafio foi inverter o fluxo lógico de pensamento para processar a string de trás para frente. Além disso, foi necessário tratar o caso em que a heurística do mau caractere sugere um recuo negativo; isso foi resolvido impondo que o salto do cursor seja sempre, no mínimo, de 1 posição, através do operador ternário: `s += (shift > 0 ? shift : 1)`.

6. Qual campo foi utilizado na criptografia?

- **Campo utilizado:** O campo *password* (Senha de Acesso) contido na estrutura binária da entidade User.
- **Justificativa:** Em conformidade com boas práticas de segurança da informação e diretrizes como a LGPD, credenciais de autenticação são categorizadas como dados altamente sensíveis. Armazená-las em texto claro (plain text) dentro de um arquivo binário .dat permitiria que qualquer usuário mal-intencionado inspecionasse o arquivo (usando ferramentas de terminal ou editores hexadecimais) e visualizasse dados confidenciais de terceiros.

7. Qual foi o método utilizado na criptografia?

O método utilizado foi a Criptografia XOR, implementada no namespace `XorCipher`. Trata-se de uma criptografia simétrica de fluxo. A função `apply` recebe o texto e realiza uma operação bit a bit de Ou Exclusivo (^) entre cada caractere do texto original e os caracteres de uma chave estática ("AEDS3-PUC-2026").